


GSOC 2026 Proposal

OPEN SCIENCE LABS (Sugar-Org)

Personal Information:

- **Name: Atharva Narayan Mhaske**
- **Email:** atharvamhaske76@gmail.com
- **Github:** @atharvamhaske
- **LinkedIn:** atharvaxdevs
- **Twitter/X:** atharvaxdevs
- **Portfolio:** atharvaxdevs.xyz
- **College:** Savitribai Phule Pune University (SPPU)
- **TimeZone:** Indian Standard Time (UTC +5:30)

Project Details:

Project Title: Implement Enterprise Security & Provider Ecosystem Expansion.

Project Size: Large (350 hrs)

Project Mentors: Sai Sanjay, Sarath Chandra, Ivan Ogasawara.

About Me:

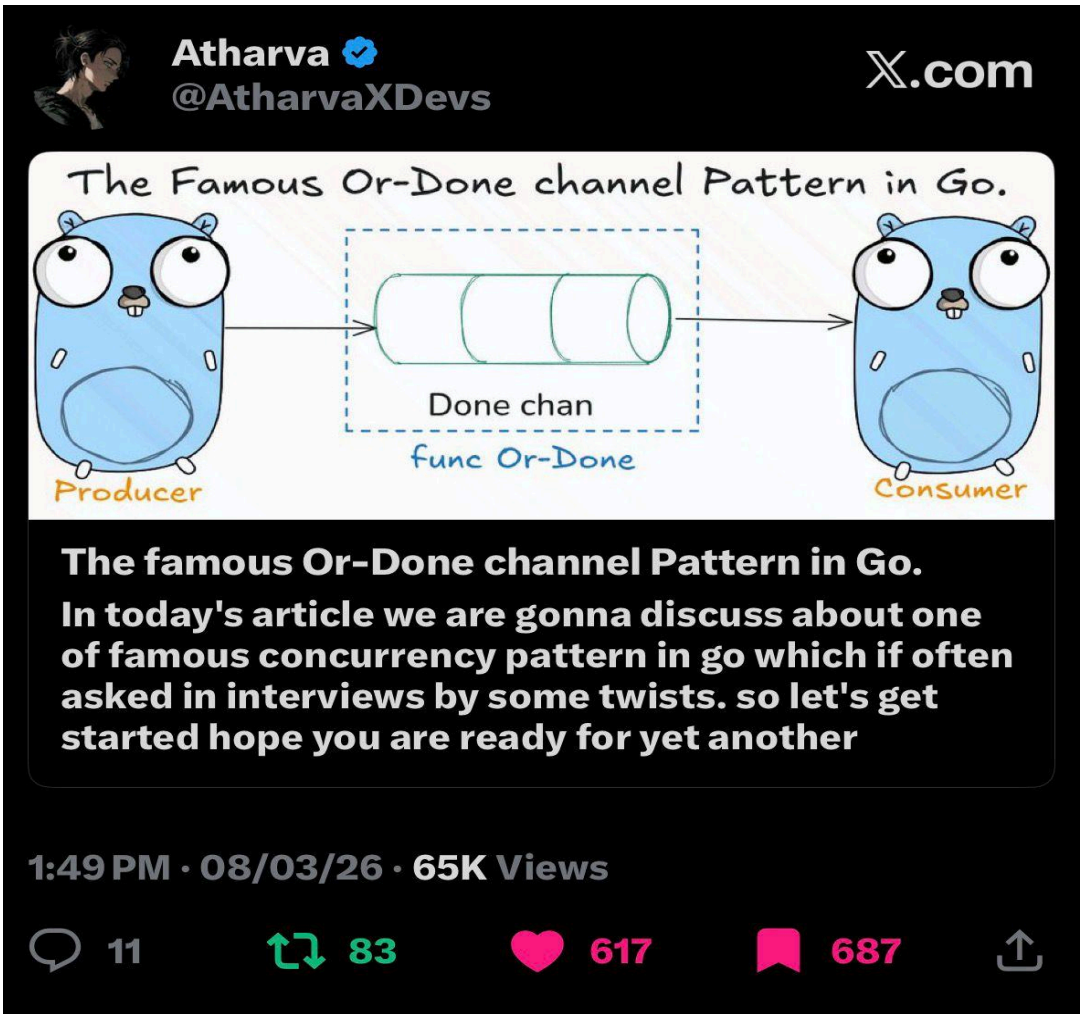
I’m Atharva Mhaske, a pre-final year student from Pune University, although my major course is Electronics, from first year itself I was more inclined towards CS as a field to pursue and work in for. I have been active in the open source community for over a year now and contributed to various projects and participated in Hacktoberfest and last term applied for **LFX ‘26** as well. I’ve been building in the Go and Devops ecosystem for over a year now, which includes a variety of projects which align with the current stack of org and future scope as well.. I have some prior experience of handling productions and taking ownership from day one by being an intern in two of startups previously. What sets me apart is I have contributed to various projects beforehand and I know the rules and etiquette of the community and how it works. Right now I'm heavily contributing in other orgs which stack mostly in Golang and cloud ecosystem and in side-questing I'm exploring databases and reading a book: DDIA.

Prior Experience:

This project revolves around the stack which I'm already familiar with and worked around in my previous work. Building Go backends for freelance clients and I have worked in CI/CD automation in some gigs for clients. This project used **Docker Swarm** and **Docker Plugins** majorly, I have explored and read all documentation regarding it along with some tech blogs. In my contributions as well I have made a contribution for migration of manual release of swarm-project to go-releaser plugin which is currently under review and will merge soon.

To wrap this up I have majorly worked across **Go, Typescript, Python, Docker, Github CI/CD** and other **Devops tools**.

I pick things up quickly and on the go. But by experience wise, I have listed everything above! I write articles on go internals and concurrency patterns as well.



One of my recent blog post is added here:

https://tinyurl.com/orDone-blog

Abstract:

Swarm external secrets is a docker plugin that bridges external secret management systems with Docker Swarm's native secret infrastructure. While the project successfully provides basic synchronization for 5 providers currently, it currently lacks several security features, authentication methods, provider integrations, and monitoring and observability capabilities that are industry standard in enterprise grade secret management solutions like the K8s based external secret operator.

This project aims to transform swarm-external-secrets into a production ready, security-hardened solutions through 3 major workstreams:

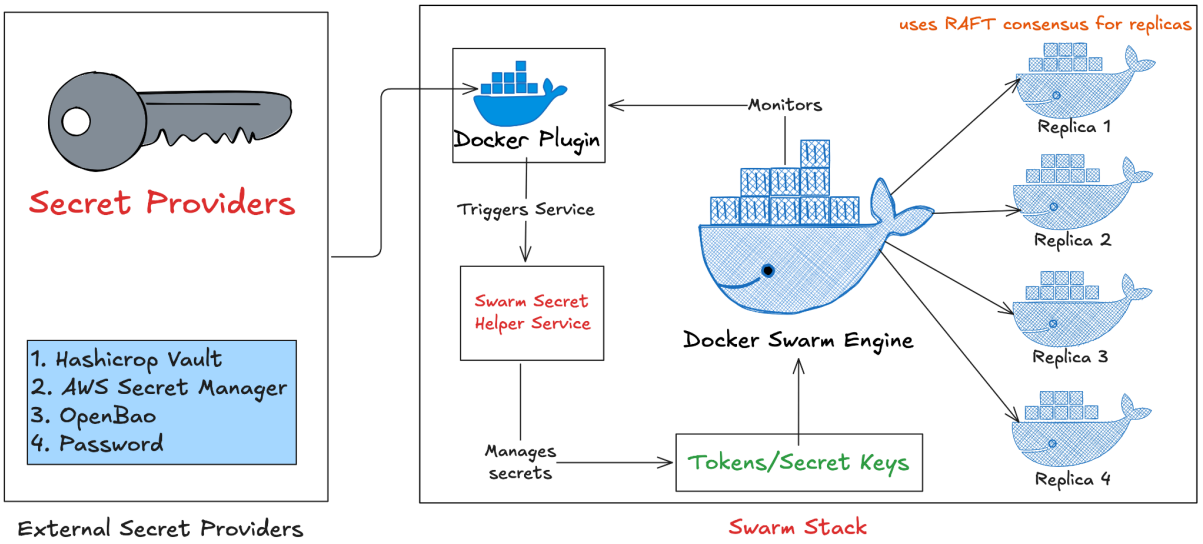
1. **Security Hardening** - Implement JWT/OIDC authentication, enhanced TLS/mTLS, CA Bundle support, and secret versioning.
2. **Push Secret Capability** - Enable bidirectional secret synchronization with update/deletion policies.
3. **New Provider Integrations** - Add 1Password, Doppler, and Infisical and other such integrations.

Optional:

- **Observability & Audit Logging** - Prometheus Metrics and Grafana for dashboard visuals, structured audit logs and health endpoints.
- **Generic Webhook provider** - Flexible HTTP based webhook integrations for all providers.

Proposed Solution:

The below is the high level overview of this project and its services used.



Architecture Overview of swarm-external-secrets

The diagram shows the high-level architecture of swarm-external-secrets, built as a Docker plugin for Docker Swarm. It sits between Swarm and

external secret providers such as HashiCorp Vault and AWS Secrets Manager. By default, Docker Swarm stores secrets in Raft logs, which is suitable for basic use cases but limited for enterprise workloads. Native Swarm secrets also make external provider integration and secret rotation harder to manage efficiently.

This plugin extends Docker through the plugin framework and communicates with Docker Engine over a Unix socket. It acts as a middleware layer that fetches secrets from external providers and returns them to Swarm on request. This makes the system provider-agnostic and allows organizations to use the backend that fits their needs. The project also includes monitoring and secret rotation support to keep secrets updated automatically.To detect changes, the plugin compares SHA-256 hashes of secret values over time.

Tradeoffs:

The current design does not cache secrets, which can increase network calls to external providers. Some provider integrations also need stronger TLS/CA-based security for production use. These can be improved through caching, stricter authentication and role policies, optimized integrations, and broader testing.

Proposed Approach -

I am keeping proposed approach for this project in three major phases:

1. **Security Hardening of existing Providers.**
2. **Addition of new secret Providers.**
3. **Observability, Monitoring & Audit Logging with Log aggregations.**

These phases are not fully rigid and can be adjusted with mentor feedback and evolving project priorities.

What I feel is before expanding support for additional providers, we should focus more on hardening the security of existing integrations to meet enterprise compliance requirements.

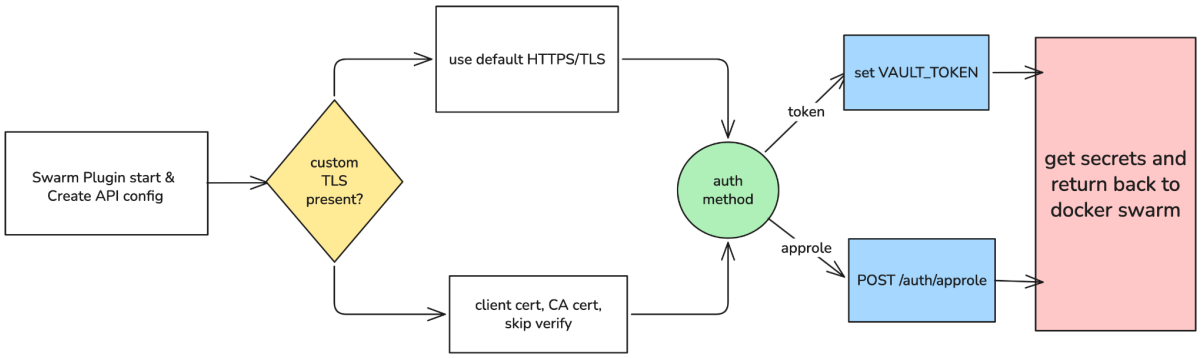
1. Security Hardening of existing Providers:

Provider	Auth methods	Current TLS	Security Gap
HashiCorp Vault	Token, AppRoles	CA + Client cert	JWT/OIDC auth
AWS Secret Manager	IAM creds, Profiles	SDK	Prefer workload identity over static creds
GCP Secret Manager	Not implemented	SDK	Stronger auth support
Azure Key Vault	Service Principal, Managed identity	SDK	Unify identity, models and configs
OpenBao	Token, Approle	CA + Client cert	JWT/OIDC auth

The existing capabilities of swarm-external-secrets and security gap within each provider are as depicted in the above diagram.

First priority would be authentication hardening, especially **JWT/OIDC**-based identity for Vault/OpenBao and workload-native identity for cloud providers, followed by **mTLS** and **CA-bundle** based transport hardening. Below shown is the current auth method used in Vault.

Today the Vault provider supports only `token` and `approle` auth in `providers/vault.go`, and rejects anything else. That means authentication depends on static pre-provisioned credentials.



Current auth for OpenBao/Vault

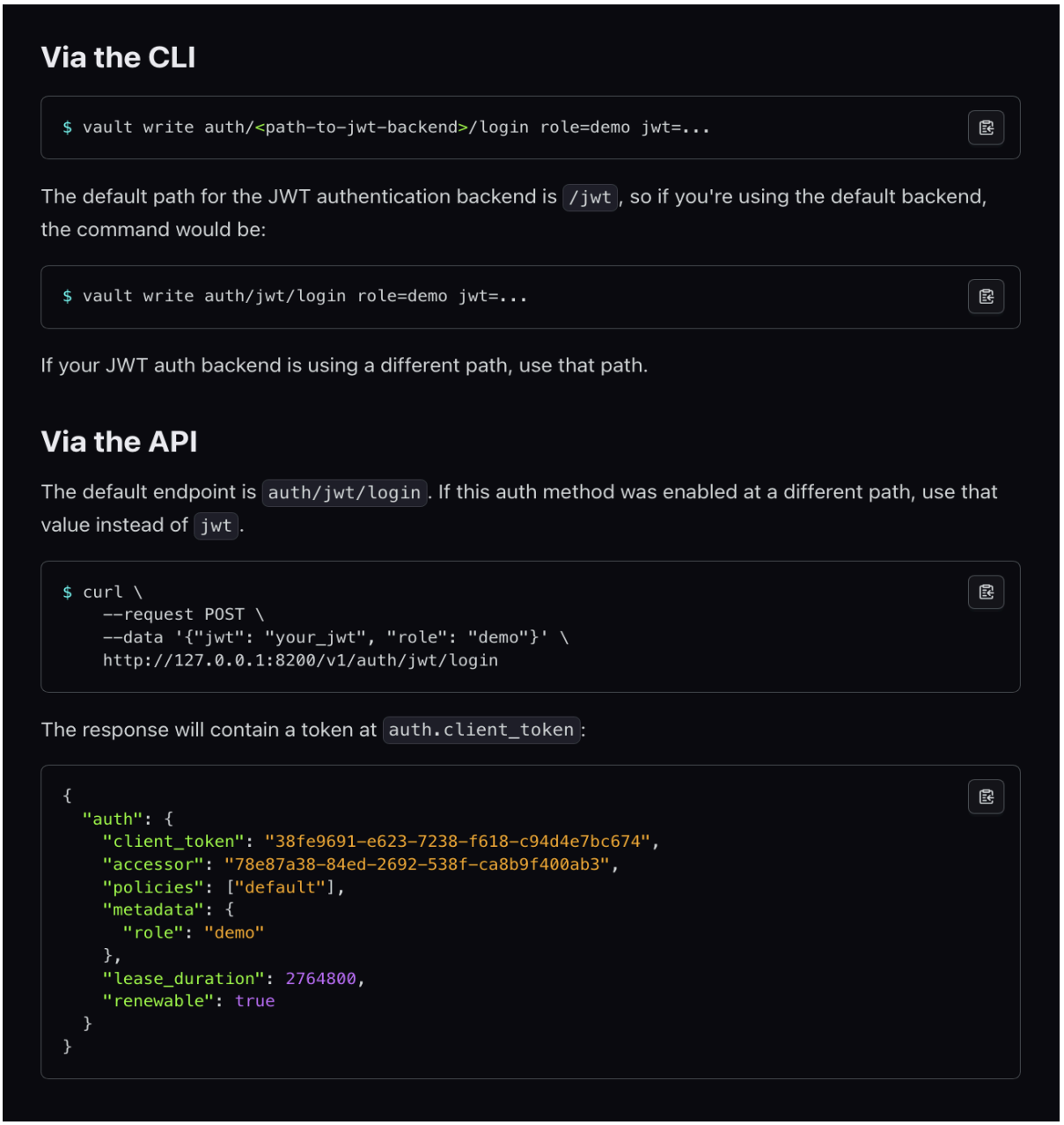
A long-lived `VAULT_TOKEN` is easy to use, but it must be stored, rotated, and protected manually. `AppRole` improves this slightly, but still depends on bootstrapped secrets like `secret_id`. In both cases, access is based on possession of a secret rather than verified workload identity.

JWT/OIDC support would close this gap by letting the plugin present a signed identity token from a trusted issuer. Vault could then validate that token and mint a short-lived Vault client token. This shifts the model from **“whoever knows the secret can log in”** to **“whoever proves identity with valid claims can log in.”** Vault supports both JWT and OIDC auth flows, and while both rely on external identity providers, they target different runtime models.

1. **JWT auth** - The JWT support model of vault is the best fit for the project because this is a headless backend plugin and here implicit flow is not available via any browser or client, Implicit flow is preferred only in such cases where we either have some client app or web ui(browser). In this model the plugin act as headless backend workload and any external IdP issues a signed JWT to do this vault provides us CLI and external API support, the best method for our project should be via CLI on route `/auth/jwt/login` the plugin presents the JWT directly to Vault at this route and we offload the validation logic on vault in the way that vault now validates the JWT using the issuer JWKS or metadata which on success issues a vault token attached policies.

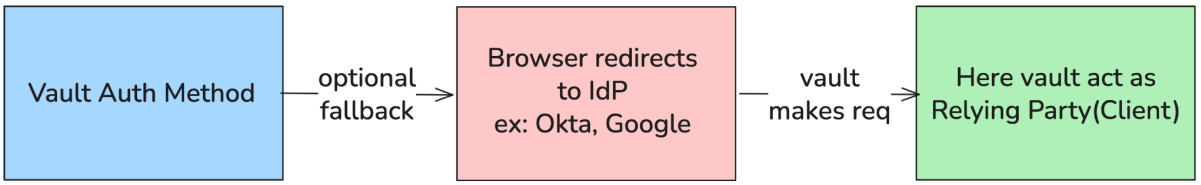
This is how it makes an important trust model -

- The IdP is the identity issue.
- Non-interaction, Machine to Machine auth w/o Browser need.
- Short Lived Credentials with Custom TTL.



Official Vault Docs for JWT support

2. OIDC : While we tend to be inclined toward JWT methods we can still use OIDC as optional interactive fallback as implicit flow as shown below.



OIDC as optional via Browser redirect

Here vault redirects the user to the IdP and the user gets authenticated in the browser tab and it returns an authorization code then vault exchanges that code for tokens.

OIDC provider configuration

The OIDC authentication flow has been successfully tested with a number of providers. A full guide to configuring OAuth/OIDC applications is beyond the scope of Vault documentation, but a collection of provider configuration steps has been collected to help get started: OIDC Provider Setup

OIDC configuration troubleshooting

This amount of configuration required for OIDC is relatively small, but it can be tricky to debug why things aren't working. Some tips for setting up OIDC:

- If a role parameter (e.g. `bound_claims`) requires a map value, it can't be set individually using the Vault CLI. In these cases the best approach is to write the entire configuration as a single JSON object:

```
vault write auth/oidc/role/demo -<<EOF
{
  "user_claim": "sub",
  "bound_audiences": "abc123",
  "role_type": "oidc",
  "policies": "demo",
  "ttl": "1h",
  "bound_claims": { "groups": ["mygroup/mysubgroup"] }
}
EOF
```

- Monitor Vault's log output. Important information about OIDC validation failures will be emitted.
- Ensure Redirect URIs are correct in Vault and on the provider. They need to match exactly. Check: http/https, 127.0.0.1/localhost, port numbers, whether trailing slashes are present.
- Start simple. The only claim configuration a role requires is `user_claim`. After authentication is known to work, you can add additional claims bindings and metadata copying.
- `bound_audiences` is optional for OIDC roles and typically not required. OIDC providers will use the client_id as the audience and OIDC validation expects this.
- Check your provider for what scopes are required in order to receive all of the information you need. The scopes "profile" and "groups" often need to be requested, and can be added by setting `oidc_scopes="profile,groups"` on the role.

Official Vault docs for OIDC

Proof of Concept:

As I am not limiting my approach only in abstract terms and buzzwords, I have already started execution of preliminary proof of concept for this JWT support for vault. The goal is simply to make clear that the plan is based on hands-on integration work and not only by reading the docs.

What the POC demonstrates ?

- Enabling and configuring Vault's JWT auth method (JWKS) and a JWT type role with bound audience and policies which is perfect as mentioned by official documentation.
- It successfully reads a KV secret after JWT login via this workflow which makes it compatible and proves that it completely fits in our architecture flow and **GetSecret()** provider lifecycle.
- This POC shows how the plugin side config(e.g. VAULT_AUTH_METHOD = jwt) performs non-interactive login without browser support. Will extend this to support fallback or optional OIDC support.

Where to find this ?

I have worked this on my local branch on my FORK so to look at approach and nitpick it please do refer following -

Repo Link - https://github.com/atharvamhaske/swarm-external-secrets

Branch Name- `poc/jwt-auth` (WIP) Demo- https://youtu.be/fzjSIuOCvCA

Limitations: This POC is intentionally narrow, full production hardening(error handling, tests, ready to handle docs) is planned to be done under project timeline after alignment of thoughts with mentor.

As this approach of security hardening via JWT/OIDC should be provider agnostic and work seamlessly with any provider we tend to add in future. Our current version is provider agnostic at the secret provider interface level but the auth is still provider specific because each provider has

different set of rules for authentication and we can't tightly couple them under a single source of truth auth flow. Unlike vault, AWS secret manager doesn't support OIDC login flow and it handles it in house through AWS creds / IAM/ SDK. I am adding my findings here, but this can be solely handled by having multiple options of case studies as well.

AWS secret manager:

In the project today AWS supports auth, but it is handled by cloud SDK's standard credential chain flow rather than through explicit pluggable auth layer like HashiCorp Vault. Here the provider in `providers/aws.go` loads config from AWSConfig struct as shown in code snippet below.

```
func (a *AWSProvider) Initialize(config map[string]string) error {
  a.config = &AWSConfig{
    Region:      getConfigOrDefault(config, "AWS_REGION", "us-east-1"),
    AccessKey:   config["AWS_ACCESS_KEY_ID"],
    SecretKey:   config["AWS_SECRET_ACCESS_KEY"],
    Profile:     config["AWS_PROFILE"],
    EndpointURL: config["AWS_ENDPOINT_URL"],
  }

  // Load AWS configuration
  cfg, err := a.loadAWSConfig()
  if err != nil {
    return fmt.Errorf("failed to load AWS config: %v", err)
  }

  // Create Secrets Manager client with optional endpoint override
  a.client = secretsmanager.NewFromConfig(cfg, func(o *secretsmanager.Options) {
    if a.config.EndpointURL != "" {
      o.BaseEndpoint = aws.String(a.config.EndpointURL)
    }
  })

  log.Printf("Successfully initialized AWS Secrets Manager provider for region: %s",
a.config.Region)
  return nil
}
```

Current Auth Config for AWS secret manager

This current integration handles baseline authentication through native SDK but it lacks an extensible identity model for modern enterprise cloud

native authentication patterns such as web identity federation and role based ephemeral access.

After researching AWS authentication patterns more deeply, I identified SPIFFE based OIDC federation as a more secure and scalable approach for this project. I documented the proof of concept design, architecture, flow, and testing workflow in an article here: [AWS attestation using SPIFFE]

The core flow in this approach is that SPIRE issues a JWT-SVID, AWS STS exchanges it through `AssumeRoleWithWebIdentity`, and the resulting short-lived credentials are used to access AWS services such as Secrets Manager. To validate this systematically, I structured the POC in three layers:

- Layer 1 verifies provider behavior and rotation logic.
- Layer 2 verifies JWT to STS to AWS access without involving the plugin.
- Layer 3 verifies the full Docker Swarm plugin path end to end.

In my current POC, I adapted this mechanism for Docker Swarm using SPIRE-issued JWT-SVIDs and validated both helper-based and direct plugin-based identity flows. As part of the project, I plan to automate the setup further, improve reliability, and strengthen the design with production-oriented validation and security hardening.

GCP secret manager:

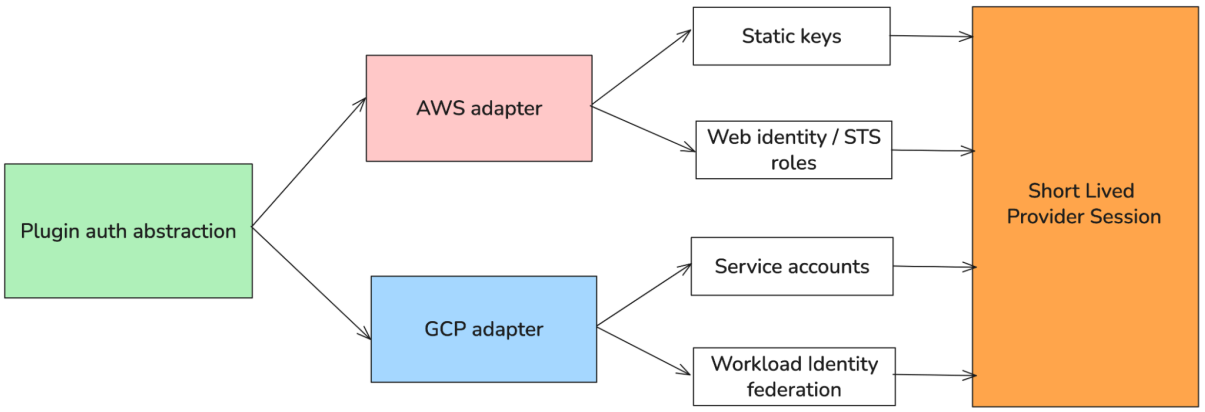
Same applies for GCP as well; the current state shows that GCP integration supports service account and ADC based auth, but it doesn't expose workload identity federation or provider agnostic auth abstraction for enterprise solutions.

For future scope the most natural direction is a secure development and can be described as:

- Extend GCP support toward workload identity federation and external account credentials.

- Extend AWS support toward web identity and STS based short lived credentials.
- Addition of structured audit logging around credential source selection, auth success/failure and token behaviour tracking.

To better visualize the enhancements described above, a diagram has been included to represent the high-level design approach.



Future Auth Scope for AWS and GCP providers.

Now in addition to mTLS support in existing workflows. Currently TLS is supported for OpenBao/Vault but there is no custom CA bundle support. TLS exists for AWS as AWS SDK uses HTTPS but custom internal PKI support is still missing, so a clean approach is to make mTLS a shared transport capability, not a provider specific feature.

Core idea:

mTLS belongs in the HTTPS/TLS client layer and in mutual TLS (mTLS), both parties authenticate each other: the client validates the server's certificate against a trusted CA, while the server validates the client's certificate against its own trusted CA. So providers should ask for a secure transport from a plugin and each provider should transport only if it actually speaks HTTPS with client cert auth.

OpenBao/Vault mTLS

Today Vault/OpenBao manually configure TLS inside each provider through runtime configs and on the other side AWS/GCP/Azure mostly relies on SDK defaults which creates duplication and inconsistency in the system. The provider agnostic model should not mean "Every provider must use mTLS" but it should mean:

- Every provider gets the same way to declare the transport service settings.
- Each provider can consume a shared TLS enabled transport mechanism.
- Providers which can't support custom mTLS configuration should just ignore that part safely and rely on the fallback method.

So keeping all points in mind the recommended config model should look like:

```
// common DTO struct
  type TransportConfig struct {
    ServerURL         string
    CABundleFile      string
    CABundlePEM       string
    ClientCertFile    string
    ClientKeyFile     string
    ClientCertPEM     string
    ClientKeyPEM      string
    InsecureSkipVerify bool
    ServerName        string
    MinTLSVersion     string
    EnableMTLS        bool
}
```

For vault you must've questioned why mTLS matters when we already have token/approle but it's a whole different thing as they protect two different things.

1. Token/Approle says that "this request had valid creds"
2. mTLS says that "this connection is from an approved workload"

```
// each provider gets
  type ProviderRuntimeConfig struct {
    ProviderName string
    Transport    TransportConfig
    Auth         map[string]string
}
```

2. Addition of new providers:

For adding a new secret provider in swarm-external-secret we must follow a simple set of rules which are already given in docs for addition of new providers, so it follows a fixed integration path to stay consistent with rotation and multi provider behaviour. In this project idea it is stated to expand the providers listed in the plugin and add new providers like Doppler, 1Password etc. So in order to validate how a new secret provider is added without disturbing existing providers flow I referred to some of the previous PRs of this project to deeply understand how it is done internally. This is how I tried implementing a preliminary Dopper integration in my own local FORK.

Current POC status:

I have made attempt to add support of Doppler provider beforehand in order to see how adding a new provider actually fits in our repo, Therefore in my own fork i have created a branch named poc/doppler to work locally on this so currently for doppler provider its plugin can be built, installed

and enabled successfully and the doppler implementation is functional at the plugin level.

However, full end to end swarm validation remains partially blocked on docker desktop due to some engine level DepedencyStore lookup failures, this needs to be debugged and fixed, i am keeping it at this stage only due to non availability of time. Once I get some time I will refer to docs again and fix this issue.

Repo: https://github.com/atharvamhaske/swarm-external-secrets

Branch:
https://github.com/atharvamhaske/swarm-external-secrets/tree/poc/doppler

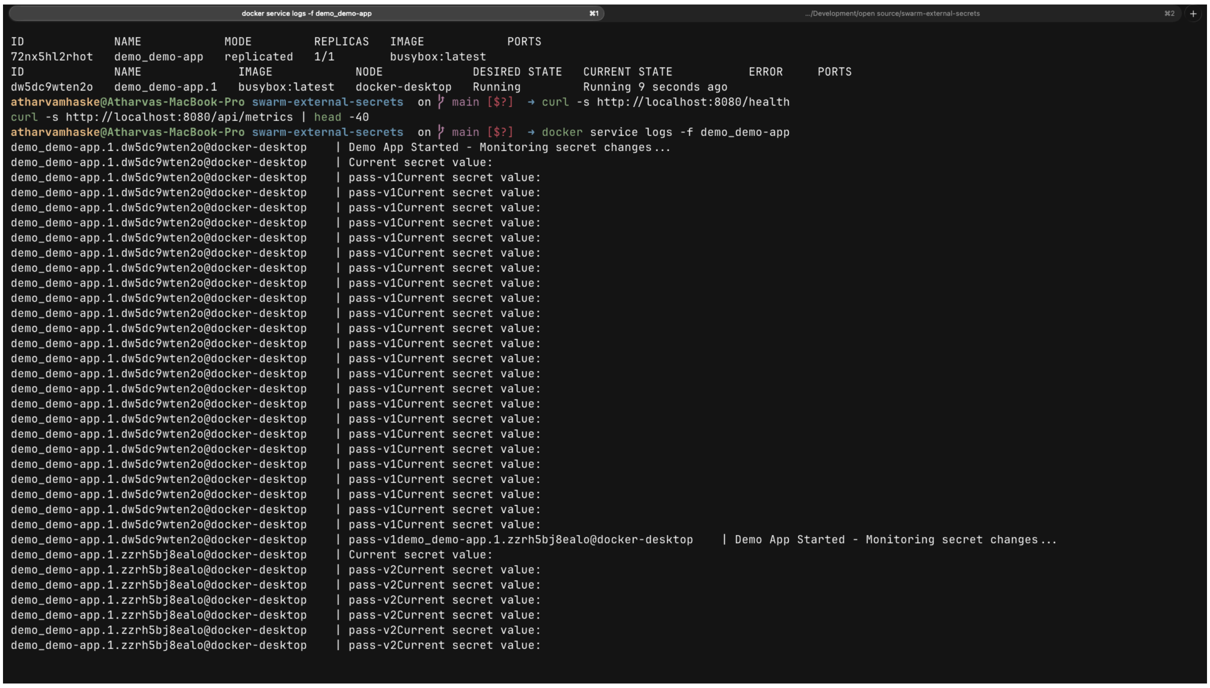
For any other providers as well like 1Password we should make sure that every provider must implement the interface in `providers/interface.go` and following methods.

Method	Purpose
Initialize(config map[string]string) error	Read env/plugin config, create API clients, validate req settings.
GetSecret(ctx, req secrets.Request) ([]byte, error)	Fetch the secret bytes for a swarm secret request.
SupportRotation() bool	True only if CheckSecretChange is implemented.
CheckSecretChanged(ctx, *SecretInfo) (bool, error)	Compare current provided value by last hashed value.
GetProviderName() string	Short name returned by the driver (“vault”, “aws”).
Close() error	Release clients if any

This is how we can add any new provider by following a simple factory design pattern.

3. Observability, Monitoring & Audit Logging with Log aggregations:

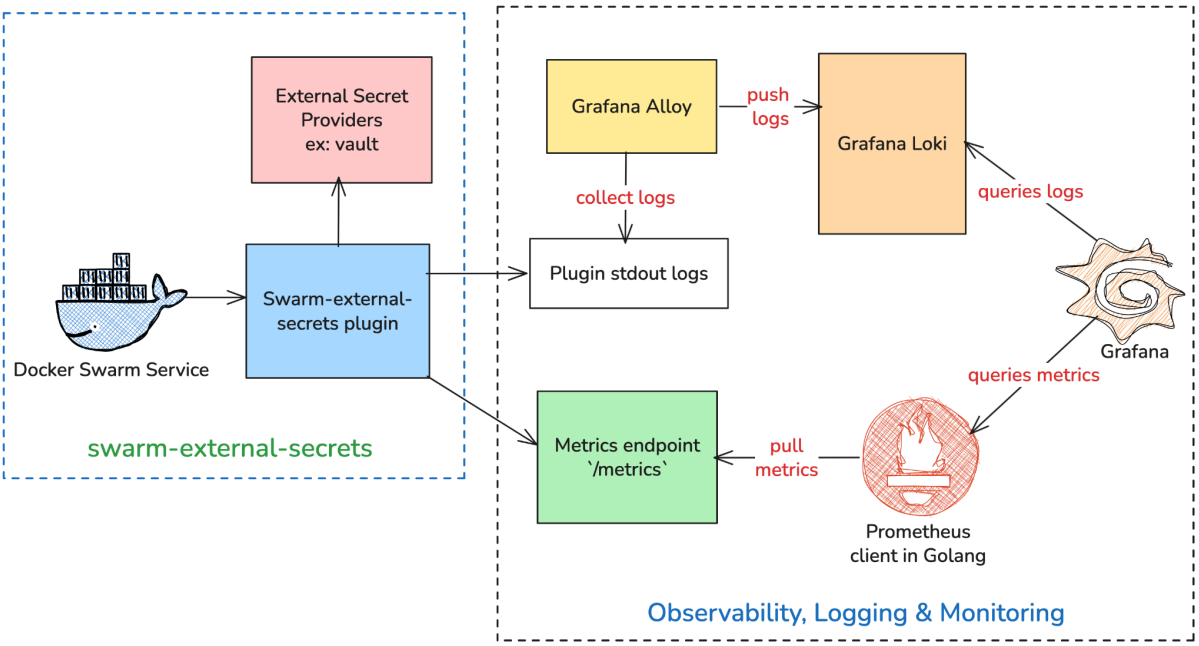
Now this final phase is very critical and important. This project needs observability and audit logging because it sits on the security path between docker swarm and external sec managers. When a secret fetch fails, a rotation misfires or anything goes wrong, the current operator experience is mostly limited to plain plugin logs, and a small in house monitoring page with basic health checks and goroutines tracking which is in the monitoring folder of this project.



Current logs are not structured and not even readable

This doesn't provide structured audit logs, provider labelled explicit metrics, request latency histograms, or dashboards and alerts which is expected in production. For compliance oriented deployments such as SOC 2, HIPPA this is a real gap, To tackle this we must use industry standard tools such as Prometheus, Grafana, Loki.

We must expose the `/metrics` endpoint in our project so that the Prometheus client can pull metrics from it, Loki for logging, Grafana for dashboards, and use of Grafana alloy instead of Promtail for log collection and shipping as promtail is deprecated now. The high level workflow architecture for this would look something like below.

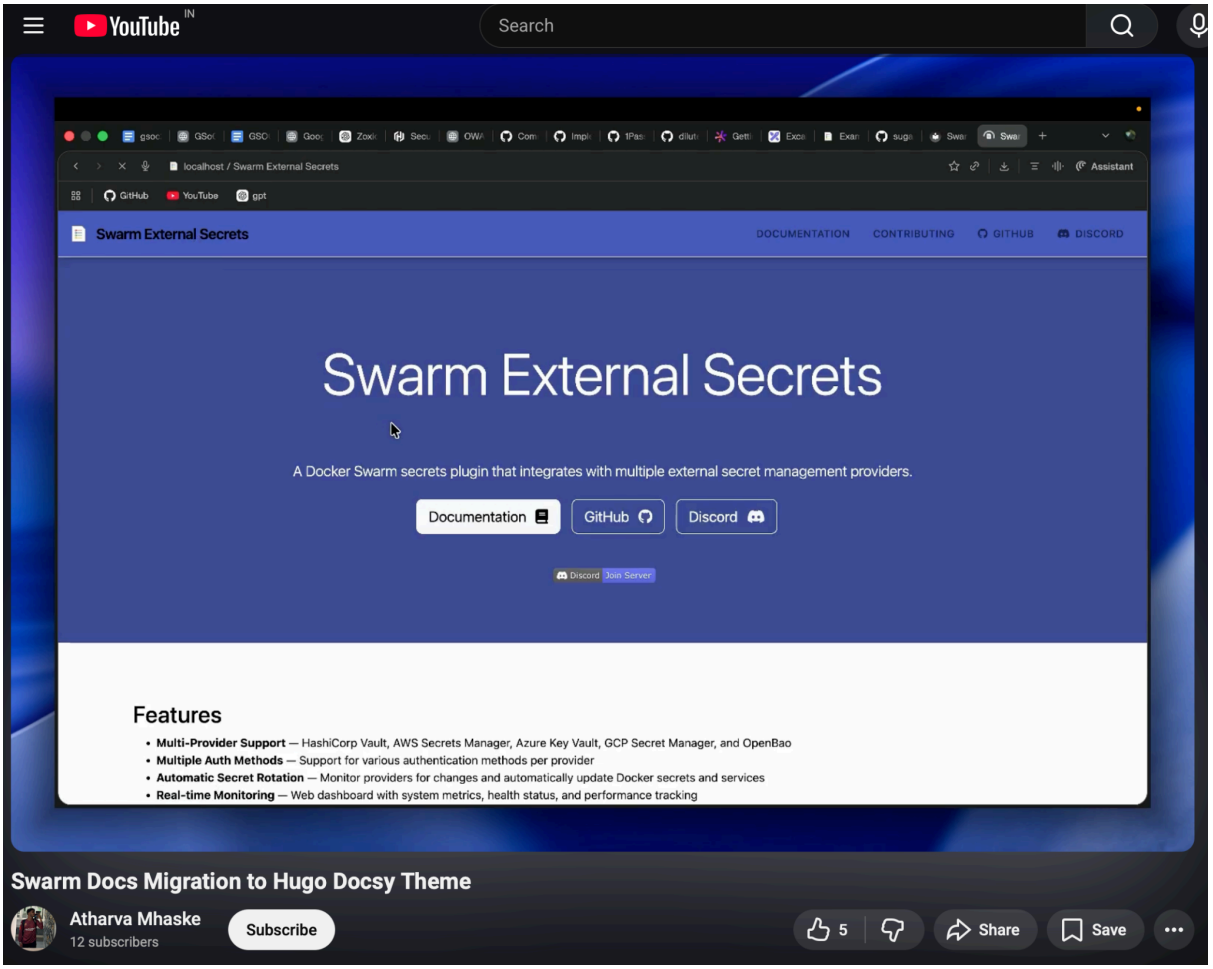


Prometheus, Grafana, Loki addition

In this architecture, the plugin continues to serve Docker Swarm secret requests while exposing operational metrics through its Prometheus-compatible `/metrics` endpoint and emitting runtime/audit logs during provider interactions, rotations, and failures. Prometheus scrapes plugin metrics, Grafana Alloy collects and forwards logs to Loki, and Grafana unifies both telemetry streams into a single operator view. This design is intentionally lightweight for a proof of concept, but it establishes the foundation for future alerting, structured audit trails, provider-specific dashboards, and security-event monitoring.

If time permits I will work on earlier proposed addons like CLI support, adding test suites and maintaining and improvisation of docs.

I have already migrated docs to docsy theme by hugo and its proof of concept demo is uploaded on my youtube channel.



Docs migration of Swarm-external-secrets

Link : https://www.youtube.com/watch?v=bg6Y6IqDgSQ

Additional Enhancements:

These are some additional improvements I would like to contribute alongside the core project work. While the primary focus would remain on the main project goals, I believe these additions would significantly improve the usability, reliability, and long-term maintainability of the project.

- **Developer CLI Experience:** The current workflow depends a lot on scripts, Makim commands, and manual Docker commands, which makes the project harder to use and onboard with. I would like to add a dedicated CLI that simplifies setup, provider selection,

configuration, and testing through a cleaner interface. A Go-based CLI using cobra, with optional TUI support through bubbletea or lipgloss, would make common tasks more guided and less error prone.

- **Centralized Logging with Loki:** Since Prometheus and Grafana are already planned for monitoring, adding Loki would make the observability stack more complete. Because this project handles sensitive secret flows and provider interactions, centralized logs would help trace failures, inspect operational history, and debug issues more effectively. I would also like to evaluate whether the current logrus usage should be improved or replaced with a more structured logging approach such as zap.
- **Unit and End-to-End Testing:** As the scope of the project expands, stronger automated testing will become increasingly important for reliability and long-term stability. I would like to improve unit testing around provider logic and also expand end-to-end coverage for real workflow validation. This would be especially valuable for areas such as secret rotation, provider-specific behavior, and cloud authentication, and tools like ginkgo and gomega could be explored where they fit well.

Project Deliverables:

For me, the minimum successful outcome here is a materially stronger and more production ready swarm-external-secrets project, hardening security of existing providers by enterprise style auth and transport level security, in addition to it will be adding the providers which are niche and on demand of users, and make sure that the project gains usable observability, audit logging and test suite coverages. The table below shows what I mean by that in more concrete terms

Deliverable	What does it mean ?
JWT/OIDC for vault	The real integration of JWT auth mode in existing workflow for OpenBao/vault where plugin can perform non-interactive auth and fallback OIDC support via browser. JWT remains as primary auth provider.
Reduce dependence on static cred	The plugin no longer relies only on long-lived VAULT_TOKEN, approle based bootstrapped secrets, instead short lived identity assertions become part of auth.
Shared TLS	CA bundle handling, client cert loading, and secure HTTPS client construction for secure and reusable transport layer instead of duplicating logic over each provider.
mTLS wherever possible	Providers that support HTTPS with client-cert auth can consume the same transport sec model, allowing the plugin to authenticate not only request but also workload identity at connection level.
CA bundle for private PKI	The plugin can trust enterprise cert authorities rather than depending only on public trust stores.
New Provider Additions	Increase provider support from 5 to at least 8-9 through existing factory patterns, for ex: Doppler, Infisical.
Docs and Demo	Creating docs and demo of each provider guiding how they can migrate from any provider and guided case studies and comparisons on choices of providers
Structured audit logging	Migrate logrus logging to standard uber zap logger and testing it manually for each function.
Push Mechanism	Discussion with mentors on the approach of push flow of

	secrets making it work bidirectionally.
Prometheus metrics	The project exposes various important metrics such as request count, failures, rotation and latencies so add a prometheus in the project to pull such metrics.
Grafana Dashboards with Loki	Dashboards for all metrics exposed via prometheus and logs can be shipped into a standard aggregation pipeline and to view alongside the dashboards with help of Grafana Loki.
End to End testing and Unit testing with optional things.	Testing the project and workflows like smoke tests and CI/CD plugins and even Unit testing via golang pkg's like ginkgo.

If these parts are completed in the right manner under mentor guidance, then for me the project is already successful and enterprise ready.

Timeline:

Time	Target
Pre GSoC [Till 29 April]	During this period I will keep contributing to the project, stay active in discussions on discord as well as any issues on github and deepen my understanding of the provider model, Docker plugin lifecycle and internal working of UNIX sockets and the existing monitoring and rotation logic. The main output from this period will be a first internal design note covering provider auth gaps, JWT integration directions and the initial shape of the deliverables.

Community Bonding Period [April 27 to May 25]	During community bonding I want to lock down the project shape before implementation begins. That means confirming the final scope of the three major workstreams, reviewing the existing provider implementations and plugin scripts in more detail, and agreeing on the exact MVP surface with mentors before opening larger implementation work. • Finalized design note and a clear implementation plan. • Setting up priority of providers and agreement on auth hardening priorities. By the end of community bonding, the project should already have a stable technical direction even if no large implementation work has started yet.
Phase 1: Security hardening of existing providers. [May 26 to June 15]	This phase focuses on authentication hardening for the currently supported providers. I will add JWT auth support for Vault, align Openbao towards the same model as it goes hand in hand and also document the existing and future identity models for AWS, GCP, Azure. The goal is to move the project from a static secret-based bootstrap pattern toward stronger identity-based flows where possible. This phase is intentionally focused on the auth side first. I don't want to add more providers before the existing ones have a stronger and more consistent security foundation.

Phase 2: Shared TLS/mTLS and transport hardening. [June 16 to June 29]	This phase focuses on transport security. I will introduce a shared TLS/mTLS abstraction for provider communication, move Vault/OpenBao away from duplicated TLS setup, and add support for CA bundles and cleaner client-cert based transport handling. The implementation will be provider-agnostic in design even if only some providers consume it fully at first. This phase is important because current TLS handling is inconsistent across providers. The goal is not to force every provider to use mTLS, but to give every provider a consistent way to declare and consume secure transport settings.
Phase 3: New provider integrations. [June 30 to July 13]	This phase focuses on validating extensibility by adding at least one new provider using the existing provider interface and factory pattern. I will use that work to ensure the project remains easy to extend even after security hardening changes are introduced. This is the point where the project moves from only improving existing providers into proving that its architecture still scales for future additions such as Doppler, 1Password, or other providers.
Midterm Evaluation [July 14 to July 18]	By midterm, I want the project to already have a meaningful security foundation completed. The expected midterm result is working JWT-based authentication for

	Vault, transport hardening groundwork in place, and at least one validated path for extending the provider set. That means even at midterm the project should already feel like a real and useful subsystem upgrade, not only preliminary scaffolding.
Phase 4: Observability, Monitoring & audit logging. [July 19 to Aug 4]	This phase focuses on making the plugin observable in a production-like way. I will improve metrics, add structured audit-style logging, and integrate a lightweight stack around Prometheus, Loki, Grafana, and Grafana Alloy. The goal is to make request failures, provider issues, and rotation behavior visible and operable. This phase is important because the plugin sits on the critical path between Docker Swarm and external secret stores. Failures in auth, transport, or rotation must be observable in a structured and actionable way.
Phase 5: Testing, Docs and demos. [Aug 5 to Aug 18]	Once auth, transport, provider integration, and observability are stable, I will tighten the test suite and improve the documentation. I will add unit tests, integration/smoke validation where practical, and improve provider/operator documentation. I will also prepare short demo material to show the hardened auth flow, provider integration, and observability stack in action.

	This phase is where the project becomes presentable and reviewable as a polished feature set rather than only an implementation branch.
Phase 6: Final polish and Mentor Feedbacks. [Aug 19 to Aug 23]	The final coding phase focuses on finishing the project cleanly. I will handle documentation gaps, refine the implementation based on mentor feedback, improve examples, and remove rough edges that remain after the main workstreams are complete. This phase also gives room for tightening tests, clarifying docs, and making the final proposal deliverables easier to review and use.
Final Evaluation [Aug 24 to Aug 31]	Final evaluation and mentor feedback also verifying that the intended project result is stronger and more enterprise ready with all deliverables delivered. Wrapping up and making GSoC dairy of daily work and blog post of overall experience.

Why Me??

These things make me right person for this project's requirements -

1. I am already familiar with relevant parts of the projects from the inside and I have made honest efforts and learned how things work under the hood. Spent enough time making my personal scripts to spin up this project in various environments to ship and test effectively.

2. My past experiences match the current stack of the project and I have worked in devops and go backends and I know how to write clean and idiomatic go with its concrete concurrency patterns.
3. Tackled moderate level issues like fixing race conditions and adding go-releaser support which was manual before.

Previous Contributions:

PR link	Description	Status
#94	bugfix: added sync.Once wrapper over Stop function which now makes it idempotent	Merged
#97	[Bug]: In driver.go checkForSecretChanges() is polling secrets in sequential order.	Open
#98	feat: separated CI workflow and added GoReleaser configuration to add release tags automated	Under Review
#106	fix: parallelize secret polling checks	Under Review
#125	[Bug]: collectMetrics() calls logMetrics() while holding m.metrics.mu write lock	Open
#POC-1	Addition of a new provider i.e., Doppler in my fork.	NA
#POC-2	This is POC implementation of attestation of AWS-SM using SPIFFE JWT SVIDs.	NA
Scripts	This repo contains all scripts i use for fasten up demo and track changes in secret rotation for swarm-external-secrets	NA

Commitments:

- During this entire summer I'm available and there are no clashes and no other commitments during summer break. I will manage to devote 35-40 hours a week to the project.
- I am not working anywhere else and I don't have any plans of taking up any other internships or other activities as I want to keep myself available for this project. I will be having my college exams in around May ending.
- I will manage that time by working extra during the holidays and weekends for some hours to complete the project within the deadline.

Extended Timeline:

- In the event of unforeseen obstacles, I am already prepared to allocate additional time to the project in subsequent weeks..
- In case of some uncertain events I am willing to work in an extended time period in case the project is not completed.

Post GSOC plans:

I am making it clear that I'm not applying for GSOC under any other organization other than this project and I have completely put all my energy into this. I am heart warmed by the mentors under this project and I really want to work with them.

Post the GSOC period, I would love to continue contributing by further adding some useful features and maintaining docs and reviewing PRs. I will be part of this community and continue putting some part of my each day for this project. I am more interested in being a long term contributor than in treating this as a one summer thing.

Conclusion:

I have tried my best in crafting this proposal and already made a plan how I would work on the current limitations, given Project Idea and addons from my end. The diagram images used in this entire proposal are all hand crafted by me on excalidraw and I have taken help of AI only for learning purpose and i have solely relied on official documentations and codebases of SDK's and all above proposed solution is my approach towards the project if something doesn't work out i am open for suggestions of mentors.
I'm looking forward to working on this project and will be glad to be onboard.

Thank you !